Project Overview: Retail Demand Forecasting & Inventory Replenishment Planner

This project provides a data-backed replenishment plan for a retail company managing products across multiple stores and warehouses. The primary objective is to **reduce stockouts** while simultaneously **controlling holding costs**.

North Star Metric

- **Service Level (Fill Rate):** The primary goal is to increase the fill rate and reduce the stockout rate to ensure customer satisfaction and capture lost sales.

Key Performance Indicators (KPIs)

The following metrics are tracked to monitor performance:

- **Forecast Accuracy:** Measured via WAPE across categories.
- **Inventory Efficiency:** Inventory turns (proxy), Days of Inventory on Hand (DOH), and overstock rates.
- **Financial Impact:** Lost sales proxy (units/₹) and holding cost proxy.
- **Operational Health:** Stockout rate and fill rate.

Data Architecture & ETL Pipeline

A reproducible Python ETL pipeline was built to clean, standardize, and transform raw data into curated datasets.

1. ETL Process

- **Standardization:** Handled missing inventory/sales data, deduplicated records, and normalized category values.
- **Outlier Detection:** Flagged unusual demand spikes.
- **Feature Engineering:** Generated promo flags, holiday flags, and calculated days of cover.

2. Curated Outputs

The pipeline generates three core files located in `/curated_dts/`:

- fact_sales_store_sku_daily.csv: Daily granular sales, revenue, and margin proxies.
- fact_inventory_store_sku_daily.csv: Daily on-hand units and stockout flags.
- replenishment_inputs_store_sku.csv: Calculated ROP, safety stock, and recommended order quantities.
- feb_sales_forecast_replenishment.csv: Projected forecasted demand and used replenishment planner on a hypothetical dataset for february 2026.

Forecasting Methodology

The system forecasts daily demand for the next **28 days** at the Store-SKU level.

- **Baseline Model:** Moving average or seasonal naive approach.
- **Advanced Model:** Random Forest Regression (including calendar, promo effects, weekend indicator, avg 28 days demand, and store wise demand)

Replenishment Policy

The replenishment logic is based on a structured reorder policy:

- **Safety Stock:** Computed based on demand variability and target service levels.
- **Reorder Point (ROP):** Calculated as: $Average\ Demand \times Lead\ Time + Safety\ Stock$.
- **Order Constraints:** Includes Minimum Order Quantities (MOQ) and shelf-life considerations for perishable SKUs.

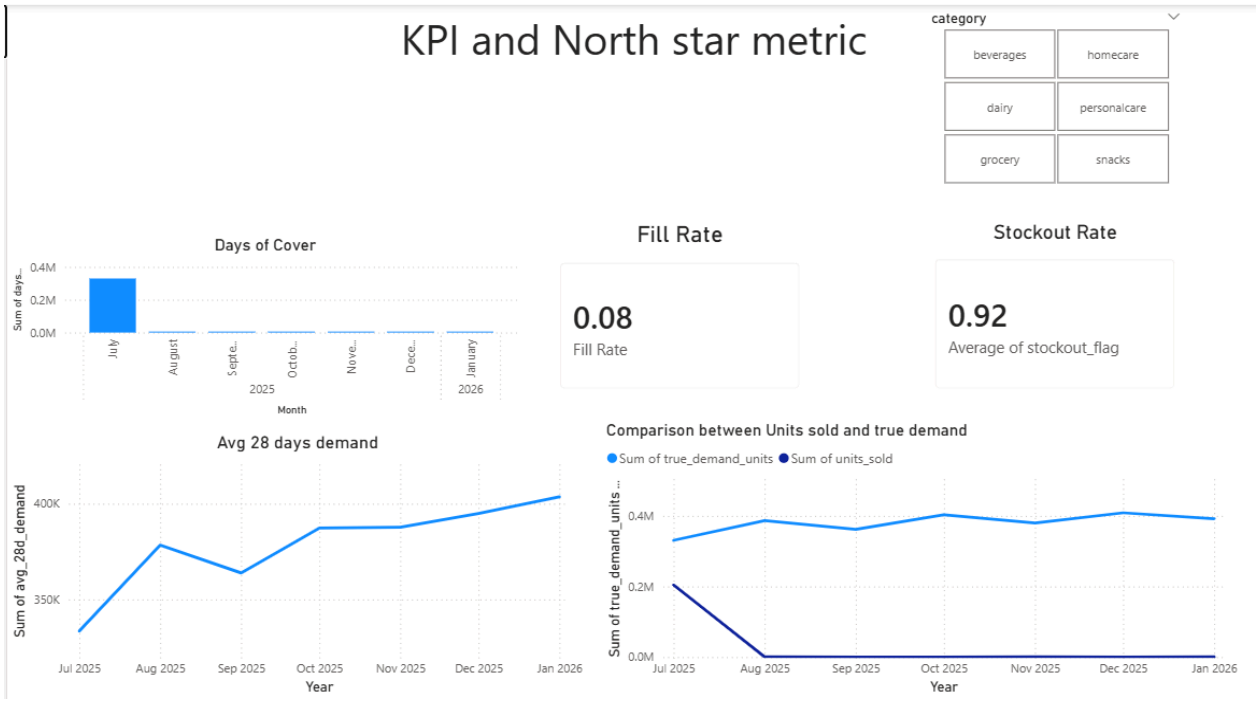
Dashboard & Analytics

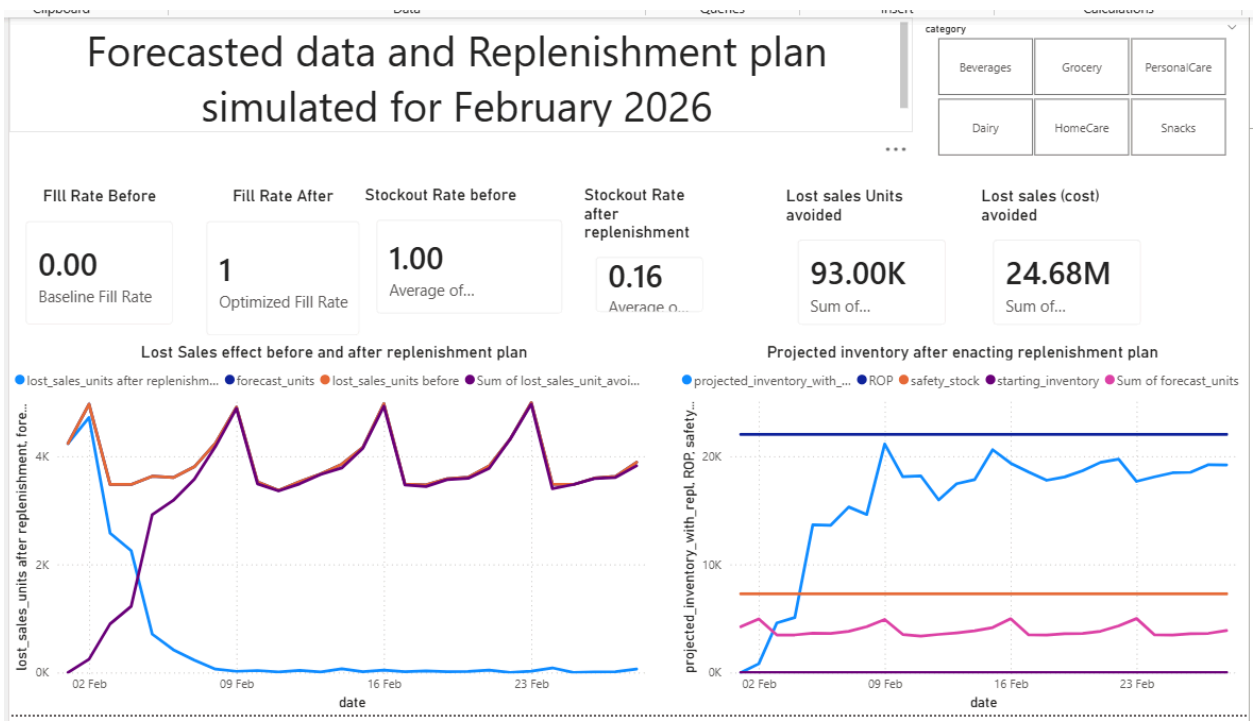
The analysis is visualized through a multi-view dashboard:

1. **Executive Summary:** High-level North Star metrics, KPI trends, and projected 30-day impact.
2. **Forecast Explorer:** Deep dive into actual vs. forecast performance by SKU and category.
3. **Replenishment Planner:** Specific reorder points and recommended PO lists for the next cycle.

\

Dashboard Screenshots:





Repository Structure

- **/etl/:** Contains **ETL_reproducible.py** for data processing and is reproducible and **ETL.ipynb** is for step by step demonstration for the ETL process
- **/analysis/**: Contains Jupyter notebooks/scripts for demand modeling and forecasting.
- **/curated_dts/**: Contains the curated output CSVs.
- **/sampler/:** Contains sample files generated from ETL_reproducible.py
- /**Power_BI_dashboard_f.pbix/**: Dashboard depicting model performance, KPIs and north star metric.
- **/Problem_statement_and_conclusion.pdf**/: Final decision memo or slide deck summarizing insights and impact.